

Construindo uma API REST Segura com Spring Security e JWT



Material Didático | Autenticação Stateless | Java 17+ | Spring Boot 3.x



Sobre este Tutorial

Neste guia, você aprenderá a proteger uma API REST Spring Boot utilizando **Spring Security** + **JWT**. Você vai construir, passo a passo, um sistema completo de autenticação e autorização com papéis (roles), pronto para ser usado em aplicações reais.

Recurso	Link
 Repositório de referência	GitHub - spring-boot-jwt-security
 Collection Postman	Download da coleção



Objetivos da Atividade

- ✓ Configurar um projeto Spring Boot com segurança.
- ✓ Criar entidades **User** e **Role** com relacionamento.
- ✓ Integrar o Spring Security à sua base de usuários via **UserDetails** e **UserDetailsService**.
- ✓ Gerar e validar tokens JWT usando a biblioteca Auth0.
- ✓ Criar um filtro personalizado que autentica requisições com o token.
- ✓ Restringir endpoints por papéis (**ADMINISTRATOR**, **CUSTOMER**).
- ✓ Testar toda a funcionalidade via Postman.



Requisitos Técnicos

Tecnologia	Versão
Java	17 ou superior
Spring Boot	3.x.x ou superior
Maven	3.6+
IDE	IntelliJ, VS Code ou Eclipse



Estrutura do Projeto

Você deve criar os pacotes e classes conforme a estrutura abaixo:

```
src/  
└─ main/
```

```
├── java/
│   └── com/
│       └── exemplo/
│           └── secrest/ (pacote raiz (exemplo: "secrest"))
│               ├── SecrestApplication.java
│               ├── controller/ (Camada de endpoints REST)
│               │   └── UserController.java
│               ├── dto/ (Objetos de transferência de dados)
│               │   ├── CreateUserDto.java
│               │   ├── LoginUserDto.java
│               │   └── RecoveryJwtTokenDto.java
│               ├── entity/ (Entidades JPA)
│               │   ├── User.java
│               │   └── Role.java
│               ├── enums/ (Enumeradores)
│               │   └── RoleName.java
│               ├── repository/ (Interfaces Spring Data JPA)
│               │   └── UserRepository.java
│               ├── security/
│               │   ├── authentication/
│               │   │   └── UserAuthenticationFilter.java (Filtro JWT)
│               │   ├── config/
│               │   │   └── SecurityConfiguration.java (Configuração
│               │               principal)
│               │   └── service/
│               │       ├── UserDetailsImpl.java (Adaptador do usuário)
│               │       ├── UserDetailsServiceImpl.java (Busca por email)
│               │       └── JwtTokenService.java (Geração/validação JWT)
│               └── service/ (Regras de negócio)
│                   └── UserService.java
└── resources/
    └── application.properties (Configuração do banco H2 e JPA)
```

 **Dica:** Mantenha essa organização desde o início. Ela facilita a localização dos arquivos e é uma boa prática profissional.

Passo 1: Criar o Projeto com as Dependências

Acesse start.spring.io e selecione:

- **Project:** Maven
- **Language:** Java
- **Spring Boot:** 3.2.x ou superior

- **Dependencies:** Spring Web, Spring Security, Spring Data JPA, H2 Database, Lombok

Após gerar e importar o projeto, adicione manualmente a dependência do JWT no `pom.xml`:

```
<dependency>
  <groupId>com.auth0</groupId>
  <artifactId>java-jwt</artifactId>
  <version>4.4.0</version>
</dependency>
```

Explicação: Essa dependência fornece as classes necessárias para criar e verificar tokens JWT (JSON Web Token) usando o padrão HMAC256. Sem ela, não é possível assinar os tokens.

Passo 2: Configurar o Banco de Dados H2

Arquivo: `src/main/resources/application.properties`

```
# Banco de dados em memória
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# Cria/atualiza tabelas automaticamente
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Habilita o console web do H2 (acessível em /h2-console)
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

Explicação: O banco H2 em memória é ideal para testes. O `ddl-auto=update` faz o Hibernate criar as tabelas automaticamente com base nas entidades. O console web permite visualizar os dados durante o desenvolvimento.

Passo 3: Criar as Entidades e o Enum

3.1. Enum `RoleName`

Caminho: `src/main/java/com/exemplo/secrest/enums/RoleName.java`

```
package com.exemplo.secrest.enums;

public enum RoleName {
    ROLE_CUSTOMER,
```

```
ROLE_ADMINISTRATOR  
}
```

Explicação: Define os papéis possíveis no sistema. O prefixo `ROLE_` é exigido pelo Spring Security para distinguir papéis de outras autoridades.

3.2. Entidade `Role`

Caminho: `src/main/java/com/exemplo/secrest/entity/Role.java`

```
package com.exemplo.secrest.entity;  
  
import com.exemplo.secrest.enums.RoleName;  
import jakarta.persistence.*;  
import lombok.*;  
  
@Entity  
@Table(name = "roles")  
@Getter  
@Setter  
@Builder  
@NoArgsConstructor  
@AllArgsConstructor  
public class Role {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    @Enumerated(EnumType.STRING)  
    private RoleName name;  
  
}
```

Explicação: A anotação `@Enumerated(EnumType.STRING)` salva o nome do enum (ex: "ROLE_CUSTOMER") no banco em vez do índice, facilitando a leitura. `@Builder` (Lombok) permite criar objetos de forma fluente.

3.3. Entidade `User`

Caminho: `src/main/java/com/exemplo/secrest/entity/User.java`

```
package com.exemplo.secrest.entity;  
  
import jakarta.persistence.*;  
import lombok.*;  
  
import java.util.List;  
  
@Entity
```

```

@Table(name = "users")
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true)
    private String email;

    private String password;

    @ManyToMany(fetch = FetchType.EAGER, cascade = CascadeType.PERSIST)
    @JoinTable(
        name = "users_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id")
    )
    private List<Role> roles;
}

```

Explicação: A associação `@ManyToMany` com `EAGER` carrega os papéis junto com o usuário em uma única consulta. O `CascadeType.PERSIST` garante que, ao salvar um novo usuário, seus papéis também sejam persistidos automaticamente.

3.4. Repository `UserRepository`

 **Caminho:** `src/main/java/com/exemplo/secrest/repository/UserRepository.java`

```

package com.exemplo.secrest.repository;

import com.exemplo.secrest.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
}

```

Explicação: O Spring Data JPA implementa automaticamente a interface. O método `findByEmail` será traduzido para uma consulta do tipo `WHERE email = ?`. O retorno `Optional` evita lidar com valores nulos.

Passo 4: DTOs (Data Transfer Objects)

 **Caminho:** dentro do pacote `dto/`

CreateUserDto.java

```
package com.exemplo.secrest.dto;

import com.exemplo.secrest.enums.RoleName;

public record CreateUserDto(
    String email,
    String password,
    RoleName role
) {}
```

LoginUserDto.java

```
package com.exemplo.secrest.dto;

public record LoginUserDto(
    String email,
    String password
) {}
```

RecoveryJwtTokenDto.java

```
package com.exemplo.secrest.dto;

public record RecoveryJwtTokenDto(
    String token
) {}
```

Explicação: `record` é um tipo imutável introduzido no Java 14/17, que gera automaticamente construtor, getters, `equals` e `hashCode`. Útil para transportar dados entre camadas sem código repetitivo.

Passo 5: Integração com Spring Security (`UserDetails`)

5.1. `UserDetailsImpl` – adaptador da entidade `User`

 **Caminho:** `src/main/java/com/exemplo/secrest/security/service/UserDetailsImpl.java`

```
package com.exemplo.secrest.security.service;

import com.exemplo.secrest.entity.User;
import lombok.Getter;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import java.util.Collection;

@Getter
public class UserDetailsImpl implements UserDetails {

    private final User user;

    public UserDetailsImpl(User user) {
        this.user = user;
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return user.getRoles()
            .stream()
            .map(role -> new SimpleGrantedAuthority(role.getName().name()))
            .toList();
    }

    @Override
    public String getPassword() {
        return user.getPassword();
    }

    @Override
    public String getUsername() {
        return user.getEmail();
    }

    // Os métodos abaixo retornam true para simplificar o exemplo
    @Override public boolean isAccountNonExpired() { return true; }
    @Override public boolean isAccountNonLocked() { return true; }
    @Override public boolean isCredentialsNonExpired() { return true; }
    @Override public boolean isEnabled() { return true; }
}
```

Explicação: O Spring Security exige que as informações do usuário estejam encapsuladas em um objeto `UserDetails`. Aqui, convertemos nossa entidade `User` para esse formato. O método `getAuthorities()` transforma cada `Role` em uma `GrantedAuthority`, que o Spring Security usa para tomar decisões de autorização.

5.2. `UserDetailsServiceImpl` – serviço que busca o usuário pelo email

Caminho:

src/main/java/com/exemplo/secrest/security/service/UserDetailsServiceImpl.java

```
package com.exemplo.secrest.security.service;

import com.exemplo.secrest.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

@Service
public class UserDetailsServiceImpl implements UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String email) throws
UsernameNotFoundException {
        var user = userRepository.findByEmail(email)
            .orElseThrow(() -> new RuntimeException("Usuário não encontrado: "
+ email));
        return new UserDetailsImpl(user);
    }
}
```

Explicação: O Spring Security chama automaticamente `loadUserByUsername` durante o processo de login. Nós implementamos essa interface para que o framework saiba como carregar os dados do usuário a partir do nosso repositório.

Passo 6: Serviço de Token JWT

 Caminho: src/main/java/com/exemplo/secrest/security/service/JwtTokenService.java

```
package com.exemplo.secrest.security.service;

import com.auth0.jwt.JWT;
import com.auth0.jwt.algorithms.Algorithm;
import com.auth0.jwt.exceptions.JWTCreationException;
import com.auth0.jwt.exceptions.JWTVerificationException;
import org.springframework.stereotype.Service;

import java.time.Instant;
import java.time.ZoneId;
import java.time.ZonedDateTime;

@Service
```



```
public class JwtTokenService {

    // ⚠ Em produção, JAMAIS hardcode secrets. Use variáveis de ambiente.
    private static final String SECRET_KEY = "4Z^XrroxR@dWxqf$mTTKwW$!@#qGr4P";
    private static final String ISSUER = "secrest-api";

    public String generateToken(UserDetailsImpl user) {
        try {
            Algorithm algorithm = Algorithm.HMAC256(SECRET_KEY);
            return JWT.create()
                .withIssuer(ISSUER)
                .withIssuedAt(creationDate())
                .withExpiresAt(expirationDate())
                .withSubject(user.getUsername())
                .sign(algorithm);
        } catch (JWTCreationException e) {
            throw new JWTCreationException("Erro ao gerar token JWT", e);
        }
    }

    public String getSubjectFromToken(String token) {
        try {
            Algorithm algorithm = Algorithm.HMAC256(SECRET_KEY);
            return JWT.require(algorithm)
                .withIssuer(ISSUER)
                .build()
                .verify(token)
                .getSubject();
        } catch (JWTVerificationException e) {
            throw new JWTVerificationException("Token JWT inválido ou expirado");
        }
    }

    private Instant creationDate() {
        return ZonedDateTime.now(ZoneId.of("America/Sao_Paulo")).toInstant();
    }

    private Instant expirationDate() {
        return
            ZonedDateTime.now(ZoneId.of("America/Sao_Paulo")).plusHours(4).toInstant();
    }
}
```

Explicação: Esse serviço concentra as operações JWT. `generateToken` cria um token com emissor, data de criação, expiração e o email do usuário como "subject". O token é assinado com HMAC256. `getSubjectFromToken` verifica a assinatura e extrai o email (subject). Em produção, a chave secreta deve vir de variável de ambiente, não ficar hardcoded.

Passo 7: Filtro de Autenticação JWT

📁 Caminho:

src/main/java/com/emplo/secrest/security/authentication/UserAuthenticationFilter.java

```
package com.emplo.secrest.security.authentication;

import com.emplo.secrest.entity.User;
import com.emplo.secrest.repository.UserRepository;
import com.emplo.secrest.security.config.SecurityConfiguration;
import com.emplo.secrest.security.service.JwtTokenService;
import com.emplo.secrest.security.service.UserDetailsImpl;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.Arrays;

@Component
public class UserAuthenticationFilter extends OncePerRequestFilter {

    @Autowired
    private JwtTokenService jwtTokenService;

    @Autowired
    private UserRepository userRepository;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws
ServletException, IOException {

        // Se o endpoint é público, não processa o token
        if (isPublicEndpoint(request)) {
            filterChain.doFilter(request, response);
            return;
        }

        String token = recoverToken(request);
        if (token != null) {
            String email = jwtTokenService.getSubjectFromToken(token);
            User user = userRepository.findByEmail(email)
                .orElseThrow(() -> new RuntimeException("Usuário não
encontrado"));
        }
    }
}
```

```

        UserDetailsImpl userDetails = new UserDetailsImpl(user);

        Authentication authentication = new
UsernamePasswordAuthenticationToken(
            userDetails.getUsername(), null,
            userDetails.getAuthorities());

        SecurityContextHolder.getContext().setAuthentication(authentication);
    } else {
        throw new RuntimeException("Token JWT não fornecido");
    }

    filterChain.doFilter(request, response);
}

private String recoverToken(HttpServletRequest request) {
    String authHeader = request.getHeader("Authorization");
    if (authHeader != null && authHeader.startsWith("Bearer ")) {
        return authHeader.replace("Bearer ", "");
    }
    return null;
}

private boolean isPublicEndpoint(HttpServletRequest request) {
    String uri = request.getRequestURI();
    return
Arrays.asList(SecurityConfiguration.ENDPOINTS_WITH_AUTHENTICATION_NOT_REQUIRED)
        .contains(uri);
}
}

```

Explicação: Este filtro é executado antes de qualquer requisição. Ele extrai o token JWT do header `Authorization`, valida-o e, se válido, cria um objeto `Authentication` com as autoridades do usuário e o coloca no `SecurityContextHolder`. Assim, o Spring Security reconhece o usuário como autenticado para a requisição atual. Endpoints públicos são ignorados.

Passo 8: Configuração Central do Spring Security

 **Caminho:** `src/main/java/com/exemplo/secrest/security/config/SecurityConfiguration.java`

```

package com.exemplo.secrest.security.config;

import com.exemplo.secrest.security.authentication.UserAuthenticationFilter;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.annotation.authentication.configuration.Authen

```

```

    ticationConfiguration;
    import org.springframework.security.config.annotation.web.builders.HttpSecurity;
    import
    org.springframework.security.config.annotation.web.configuration.EnableWebSecurity
    ;
    import org.springframework.security.config.http.SessionCreationPolicy;
    import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
    import org.springframework.security.crypto.password.PasswordEncoder;
    import org.springframework.security.web.SecurityFilterChain;
    import
    org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

    @Configuration
    @EnableWebSecurity
    public class SecurityConfiguration {

        // ===== ENDPOINTS PÚBLICOS =====
        public static final String[] ENDPOINTS_WITH_AUTHENTICATION_NOT_REQUIRED = {
            "/users/login",
            "/users"
        };

        // ===== ENDPOINTS COM ACESSO RESTRITO =====
        public static final String[] ENDPOINTS_ADMIN = {
            "/users/test/administrator"
        };

        public static final String[] ENDPOINTS_CUSTOMER = {
            "/users/test/customer"
        };

        @Autowired
        private UserAuthenticationFilter userAuthenticationFilter;

        @Bean
        public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
        Exception {
            return http
                .csrf(csrf -> csrf.disable())
                .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authorizeHttpRequests(authorize -> authorize

                .requestMatchers(ENDPOINTS_WITH_AUTHENTICATION_NOT_REQUIRED).permitAll()
                .requestMatchers(ENDPOINTS_ADMIN).hasRole("ADMINISTRATOR")
                .requestMatchers(ENDPOINTS_CUSTOMER).hasRole("CUSTOMER")
                .anyRequest().authenticated()
                )
                .addFilterBefore(userAuthenticationFilter,
                UsernamePasswordAuthenticationFilter.class)
                .build();
        }
    }

```

```

@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration
authConfig) throws Exception {
    return authConfig.getAuthenticationManager();
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
}

```

Explicação: A configuração desabilita CSRF (adequado para APIs stateless), define política de sessão como **STATELESS** (sem sessão HTTP) e estabelece as regras de autorização: endpoints públicos, endpoints apenas para ADMIN, apenas para CUSTOMER, e qualquer outra requisição exige autenticação. O **AuthenticationManager** gerencia o processo de login. O **PasswordEncoder** com **BCrypt** codifica as senhas antes de salvá-las.

Passo 9: Service e Controller

9.1. **UserService** – lógica de negócio

 **Caminho:** `src/main/java/com/emplo/secrest/service/UserService.java`

```

package com.exemplo.secrest.service;

import com.exemplo.secrest.dto.CreateUserDto;
import com.exemplo.secrest.dto.LoginUserDto;
import com.exemplo.secrest.dto.RecoveryJwtTokenDto;
import com.exemplo.secrest.entity.Role;
import com.exemplo.secrest.entity.User;
import com.exemplo.secrest.repository.UserRepository;
import com.exemplo.secrest.security.service.JwtTokenService;
import com.exemplo.secrest.security.service.UserDetailsImpl;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class UserService {

    @Autowired
    private AuthenticationManager authenticationManager;

```

```

@Autowired
private JwtTokenService jwtTokenService;

@Autowired
private UserRepository userRepository;

@Autowired
private PasswordEncoder passwordEncoder;

public RecoveryJwtTokenDto authenticateUser(LoginUserDto loginDto) {
    var authToken = new UsernamePasswordAuthenticationToken(loginDto.email(),
loginDto.password());
    Authentication authentication =
authenticationManager.authenticate(authToken);
    UserDetailsImpl userDetails = (UserDetailsImpl)
authentication.getPrincipal();
    String token = jwtTokenService.generateToken(userDetails);
    return new RecoveryJwtTokenDto(token);
}

public void createUser(CreateUserDto createDto) {
    User newUser = User.builder()
        .email(createDto.email())
        .password(passwordEncoder.encode(createDto.password()))
        .roles(List.of(Role.builder().name(createDto.role()).build()))
        .build();
    userRepository.save(newUser);
}
}

```

Explicação: `authenticateUser` cria um token de autenticação com email/senha, delega ao `AuthenticationManager` (que usa o `UserDetailsServiceImpl` para validar as credenciais) e, em caso de sucesso, gera o JWT. `createUser` codifica a senha com BCrypt antes de salvar e associa um único papel ao usuário (pode ser ajustado para múltiplos).

9.2. UserController – endpoints REST

 **Caminho:** `src/main/java/com/exemplo/secrest/controller/UserController.java`

```

package com.exemplo.secrest.controller;

import com.exemplo.secrest.dto.CreateUserDto;
import com.exemplo.secrest.dto.LoginUserDto;
import com.exemplo.secrest.dto.RecoveryJwtTokenDto;
import com.exemplo.secrest.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController

```

```
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserService userService;

    @PostMapping
    public ResponseEntity<Void> createUser(@RequestBody CreateUserDto dto) {
        userService.createUser(dto);
        return ResponseEntity.status(HttpStatus.CREATED).build();
    }

    @PostMapping("/login")
    public ResponseEntity<RecoveryJwtTokenDto> login(@RequestBody LoginUserDto
dto) {
        RecoveryJwtTokenDto token = userService.authenticateUser(dto);
        return ResponseEntity.ok(token);
    }

    @GetMapping("/test")
    public ResponseEntity<String> test() {
        return ResponseEntity.ok("Autenticado com sucesso!");
    }

    @GetMapping("/test/customer")
    public ResponseEntity<String> customerTest() {
        return ResponseEntity.ok("Acesso de CUSTOMER autorizado!");
    }

    @GetMapping("/test/administrator")
    public ResponseEntity<String> adminTest() {
        return ResponseEntity.ok("Acesso de ADMINISTRATOR autorizado!");
    }
}
```

Explicação: O controller expõe os endpoints. Os métodos `test`, `customerTest` e `adminTest` servem apenas para verificar se a autenticação e as permissões estão funcionando. A anotação `@RestController` já implica `@ResponseBody` em todos os métodos.



Passo 10: Testando a API

10.1. Inicie a aplicação

```
mvn spring-boot:run
```

10.2. Crie um usuário com papel `ROLE_CUSTOMER`

Endpoint: `POST http://localhost:8080/users`

```
{
  "email": "maria@email.com",
  "password": "123456",
  "role": "ROLE_CUSTOMER"
}
```

✓ Resposta esperada: 201 CREATED

10.3. Faça login

Endpoint: POST `http://localhost:8080/users/login`

```
{
  "email": "maria@email.com",
  "password": "123456"
}
```

✓ Resposta esperada (exemplo):

```
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." }
```

10.4. Acesse um endpoint protegido

Adicione o header **Authorization** com o valor `Bearer SEU_TOKEN_JWT`.

- **Endpoint CUSTOMER:** GET `http://localhost:8080/users/test/customer` → ✓ acesso liberado
- **Endpoint ADMIN:** GET `http://localhost:8080/users/test/administrator` → ✗ retorna 403 Forbidden

10.5. Crie um usuário ADMIN e repita os testes

Basta alterar o `role` para `ROLE_ADMINISTRATOR` no cadastro. O usuário administrador terá acesso a ambos os endpoints restritos.

Desafio Prático (para você fazer)

Agora que você tem a base funcionando, que tal estender o projeto com uma funcionalidade real? O desafio é:

✨ Adicionar um endpoint GET `/users/me` que retorna os dados do próprio usuário autenticado.

Objetivo

- Reforçar o uso do token JWT para identificar o usuário logado.

- Praticar a injeção do `UserDetails` no controller.

Requisitos

1. O endpoint deve retornar o **id**, **email** e **lista de roles** do usuário autenticado.
2. Deve ser acessível apenas para usuários com **qualquer papel** (qualquer token válido).
3. Formato de resposta sugerido:

```
{
  "id": 1,
  "email": "maria@email.com",
  "roles": ["ROLE_CUSTOMER"]
}
```

Passos sugeridos

1. **Criar um DTO de resposta** `UserProfileDto` dentro do pacote `dto`:

```
public record UserProfileDto(Long id, String email, List<String> roles) {}
```

2. **No `UserController`**, adicione o novo endpoint:

```
@GetMapping("/me")
public ResponseEntity<UserProfileDto> getCurrentUser(Authentication
authentication) {
    // O Spring Security já injeta o objeto Authentication automaticamente
    String email = authentication.getName();
    User user = userRepository.findByEmail(email)
        .orElseThrow(() -> new RuntimeException("Usuário não encontrado"));
    List<String> roles = user.getRoles().stream()
        .map(role -> role.getName().name())
        .toList();
    return ResponseEntity.ok(new UserProfileDto(user.getId(), user.getEmail(),
roles));
}
```

3. **Injete o `UserRepository` no controller** (ou faça via `UserService` – escolha a melhor prática).
4. **Configure a segurança** no `SecurityConfiguration` para permitir acesso autenticado ao novo endpoint:

```
.requestMatchers("/users/me").authenticated()
```

5. **Teste** com um token válido: `GET /users/me` deve retornar os dados do usuário.

Dica extra

Se quiser ir além, faça com que o **UserService** tenha um método **findByEmail** e use-o no controller. Isso separa melhor as responsabilidades.

Conclusão

Parabéns! Agora você possui uma API REST completa com Spring Security e JWT, aprendeu a estruturar um projeto Java profissional e ainda completou um desafio prático. Esse conhecimento é a base para sistemas mais complexos, como microsserviços e aplicações com front-end separado.

Próximos passos (estudos opcionais):

- Implementar **Refresh Token** para aumentar a segurança.
 - Substituir o banco H2 por **MySQL/MongoDB** (ou outro banco real).
 - Adicionar **Swagger/OpenAPI** para documentar a API.
 - Fazer o deploy no **Railway** ou **Heroku**.
-